

Methodology Appendix

AI-Assisted Development and Verification

How `creditlib` was specified, generated, and independently validated

Shreyas Sundaresan

Development methodology and model-risk record

Purpose. This appendix documents that `creditlib` was built with substantial use of a large language model, and specifies the verification architecture that makes the resulting code trustworthy.

Position. The relevant question about AI-assisted quantitative development is not whether it was used. It is whether the person directing it can specify the problem correctly, detect what the model gets wrong, and prove the output is right by means independent of the tool that produced it. This document is the record of those three things.

Summary. Six defects were found and corrected. Three were surfaced by validation against an independent oracle; two by the author running the engine on data it had not been developed against; one by the author rejecting an incorrect claim in a draft. None were surfaced by the model reviewing its own work.

August 2026

Contents

1	Why Disclose	2
1.1	The alternative is worse	2
1.2	What the disclosure is meant to establish	2
2	The Development Protocol	2
2.1	Derivation before implementation	2
2.2	Independent oracle	2
2.3	Structural assertions, not just numerical ones	3
2.4	Incremental build with comprehension gates	3
3	Division of Labour	3
4	Defect Record — What Caught What	4
4.1	Defect 1 — Inverted day-count direction	4
4.2	Defect 2 — RPV01 scaled by notional	4
4.3	Defect 3 — Wrong maturity convention	4
4.4	Defect 4 — Bootstrapping off the dirty par spread	4
4.5	Defect 5 — Clean par spread against dirty mark-to-market	4
4.6	Defect 6 — An incorrect claim in a draft	5
4.7	Attribution summary	5
5	Architecture as Evidence	5
6	Limitations of the Approach	6
7	Independent Verification	6

1 Why Disclose

1.1 The alternative is worse

Quantitative development with LLM assistance is now common practice at trading desks, risk functions and rating agencies. Presenting a project of this size without acknowledging it would be either misleading or, if believed, an invitation to a technical interview the candidate could not survive.

More practically: the entire credibility of this project rests on a claim about *verification* — 104 tests, parity with QuantLib to 10^{-12} , a documented defect log. A verification claim that omits how the code was produced is incomplete.

1.2 What the disclosure is meant to establish

The skill being evidenced is not code generation. It is **specification, detection and independent validation** — the ability to state a problem precisely enough that generated output can be checked, to notice when it is wrong, and to prove correctness by a route that does not share a failure mode with the generator.

That is the same discipline a model validation function applies to a model built by a front-office quant. The tooling is new; the control framework is not.

2 The Development Protocol

Four constraints were imposed at the outset and held throughout. Each exists to make generated output falsifiable.

2.1 Derivation before implementation

No formula was accepted as an assertion. Every expression in the primary documentation is obtained from a stated definition or a replication argument before any corresponding code was written — the hazard rate from the conditional-probability limit, forward rates from a no-arbitrage replication portfolio, the protection-leg integral in closed form.

This is the principal control against plausible-but-wrong output. A language model will produce a confident, well-formatted, incorrect formula. It is markedly harder for it to produce a confident, well-formatted, incorrect *derivation* that survives being read line by line — and impossible for the error to survive a derivation the reader has reproduced.

The resulting 25-page methodology document is not decoration. It is the audit trail that makes the code checkable by a third party.

2.2 Independent oracle

QuantLib was used exclusively as an external check and never appears on the pricing path. This matters more than it appears: a validation suite written by the same process that wrote the code shares its blind spots. QuantLib is a mature, widely used library developed entirely independently, so agreement to 10^{-12} is evidence, not self-congratulation.

Where conventions differ — QuantLib's `fairSpread()` uses the dirty annuity, its `ZeroCurve` interpolates linearly on zero rates — the discrepancy is quantified and explained rather than absorbed into a loose tolerance.

2.3 Structural assertions, not just numerical ones

Tests assert economic properties, not merely that numbers match previously observed numbers:

- Survival probabilities must lie in $(0, 1]$ and be non-increasing for any input.
- A negative hazard, or a rising survival curve, must be rejected at construction.
- Par-spread convergence to the credit triangle $h(1 - R)$ as premium frequency rises.
- Bumping the 10Y quote must leave $h_1 \dots h_4$ bit-for-bit unchanged — the triangular structure of the bootstrap, asserted with `np.array_equal`.

A regression-only suite would have caught none of the convention errors in Section 4.

2.4 Incremental build with comprehension gates

The engine was constructed in six specified layers, each fully validated before the next was started. Each layer required the derivation to be understood before implementation proceeded.

The purpose of the gates was to prevent the accumulation of unexamined code. A system generated in one pass is unauditable regardless of who or what generated it; the failure mode is that nobody can say why any particular line is there.

3 Division of Labour

Stated plainly, because a vague account of this is worse than none.

Author	LLM
Project specification and the two binding constraints (derive first; validate against no-arbitrage and QuantLib)	Derivations, implementation, test construction, documentation drafting
Layer sequencing and scope decisions	Proposal of algorithms and closed forms within each layer
All market inputs: bank spreads, issuer fundamentals, sector classifications	Numerical methods — bisection over Newton, piecewise-flat interpolation, exact integration over quadrature
Fundamental credit judgement: the HTM thesis, the AT1/preferred conclusion, TLAC treatment of bank holdco senior	LaTeX, chart and presentation generation
Requirement to endogenise recovery; requirement to scale to twenty issuers	Identification of the $h(1 - R)$ degeneracy and the three routes to break it
Execution on independent infrastructure; detection of two defects thereby	—
Rejection of an incorrect claim in a draft (Section 4, defect 6)	—

The honest summary: **the code was generated; the specification, the market judgement, the verification standard and the error detection were not.** Those are the parts that transfer to a desk, and they are the parts this document evidences.

4 Defect Record — What Caught What

Six defects reached working code and were corrected. The attribution matters more than the count.

4.1 Defect 1 — Inverted day-count direction

Found by: a structural assertion.

An assertion that ACT/360 produces a higher quoted rate than ACT/365F failed. ACT/360 yields the *larger* year fraction ($91/360 = 0.25278$ against 0.24932), so for identical growth the quoted simple rate is *lower*. The intuition that “divide by 360 shrinks the fraction” is wrong, and the test caught it.

4.2 Defect 2 — RPV01 scaled by notional

Found by: comparison against QuantLib conventions.

The risky annuity was reported multiplied by notional rather than per unit. Mathematically harmless — it cancels in the par spread — but against market convention and misleading on output.

4.3 Defect 3 — Wrong maturity convention

Found by: the independent oracle.

Schedules measured tenor from the trade date, making every contract up to a full quarter too long: a “5Y” maturing 20 September 2031 against QuantLib’s 20 June 2031. The CDS2015 convention measures tenor from the *accrual start*, the preceding IMM date. The error produced a 66bp distortion in the bootstrapped 1Y hazard.

Nothing internal to the code was inconsistent. Only an external reference exposed it.

4.4 Defect 4 — Bootstrapping off the dirty par spread

Found by: the independent oracle, and diagnosed via a previously derived decomposition.

On a seasoned schedule the annuity carries a coupon stub for which no protection is provided. On a 0.84-year contract that stub is 16% of the annuity, inflating the front hazard by 18bp — roughly 20% wrong at the short end. Netting the accrued rebate collapsed the gap:

	Dirty	Clean	QuantLib	Clean – QL
1Y	110.38	92.44	92.14	+0.31 bp
3Y	162.46	158.44	158.47	−0.03 bp
10Y	284.22	282.31	282.28	+0.04 bp

4.5 Defect 5 — Clean par spread against dirty mark-to-market

Found by: the author, running the engine on independent infrastructure.

Correcting defect 4 introduced an inconsistency: the par spread used the clean annuity while the mark-to-market continued to use the dirty one. The identity $MTM = (s_{\text{par}} - s)AN$ therefore failed by exactly the accrued rebate, and pricing *at* the par spread returned $-\text{accrued}$ rather than zero.

This defect survived 79 passing tests including exact QuantLib parity. Every MTM test used a synthetic schedule with periods of exactly 0.25 years — convenient, because it strips out the calendar. But the accrued stub *is* the calendar: on that fixture the clean and dirty annuities are numerically identical, so the inconsistency was structurally invisible. The QuantLib comparison missed it too, because it had been made against `fairSpread()`, which is itself the dirty convention.

It was found by executing the engine in a clean Colab environment on a real trade date and noticing that two printed figures did not reconcile. No automated check would have caught it; the blind spot was in the fixture design, and fixture design is not something a test suite can audit.

The generalisable lesson, now recorded in the primary documentation: *convenient test fixtures are usually degenerate in exactly the dimension that was simplified away*. Seven regression tests on a deliberately seasoned fixture now cover it.

4.6 Defect 6 — An incorrect claim in a draft

Found by: the author, rejecting a stated conclusion.

A draft asserted that the recovery assumption drove the *ranking* of implied default probabilities. It does not. Under a uniform recovery, $h = s/(1 - R)$ is a monotone transform of the spread, so the ordering is invariant; recovery moves levels only. The claim was rejected, the analysis rerun, and the corrected finding — that only *name-specific* recovery can reorder — became the central result of the recovery extension.

4.7 Attribution summary

Detection route	Defects
Structural assertion in the test suite	1
Independent oracle (QuantLib)	2, 3, 4
Author executing on independent infrastructure	5
Author rejecting a stated conclusion	6
Model reviewing its own output	0

No defect was caught by the generating model reviewing its own work. This is the central finding of the appendix and the reason the verification architecture exists. Self-review is not a control. Independent references, structural assertions and execution on unfamiliar data are.

5 Architecture as Evidence

One structural fact serves as an independent check on whether the design was understood rather than merely produced.

The recovery model — a new module, a new class hierarchy, an additional layer of economic reasoning — was added late in the project. It required **no change to any pricing code**. The bootstrap already accepted recovery as an argument, so all 86 pre-existing tests passed unmodified.

A system generated without design intent does not have that property. Separation of concerns is either present from the start or absent permanently, and it is only visible when something new is bolted on.

6 Limitations of the Approach

Stated because a methodology document that lists only strengths is not a methodology document.

1. **Verification cannot exceed the specification.** Errors in *what* was asked for propagate silently. The recovery model's coefficients are calibrated to published anchors rather than estimated from data — a limitation of scope, not of implementation, and one no amount of testing would surface.
2. **The oracle constrains what is checkable.** QuantLib covers standard CDS pricing. It offers no independent check on the recovery model, which is why that module carries a correspondingly heavier caveat and an explicit validation plan rather than a validation result.
3. **Generation speed exceeds comprehension speed.** The layered build with comprehension gates was imposed precisely because this mismatch is the principal risk of the method. It is a mitigation, not a solution.
4. **Fixture blind spots are not self-detecting.** Defect 5 is the proof. A test suite cannot audit the representativeness of its own fixtures — only execution on unfamiliar data can.
5. **This document is self-reported.** The verifiable claims are the repository, the test suite and the QuantLib comparison, all of which a reader can execute independently. Section 7 explains how.

7 Independent Verification

Every claim in this appendix is checkable without taking anything on trust. In a clean environment:

```
git clone https://github.com/Shreyassundaresan/creditlib.git
cd creditlib
pip install -e ".[test]"
pytest -q                      # expect: 104 passed
python quickstart.py           # sanity anchors printed at the end
```

The QuantLib parity tests are in `tests/`, marked by name, and skip cleanly if QuantLib is unavailable. The defect log is Section 7.2 of the primary documentation. The derivations precede every implementation in the same document.

The appropriate posture toward this project is not to take the disclosure on faith in either direction. Clone it, run the suite, read a derivation, and check whether the code implements what the mathematics says.

Market spreads are indicative and supplied as inputs; discount curves are illustrative. Recovery estimates are output from calibrated coefficients and have not been validated against realised recoveries. Nothing in this document is investment advice.